

# Vedant Desai

(248) 704 – 4852 | [vedantde@umich.edu](mailto:vedantde@umich.edu) | [github.com/Verdent06](https://github.com/Verdent06) | [linkedin.com/in/vedantde06](https://linkedin.com/in/vedantde06)

## EDUCATION

### University of Michigan

B.S. in Computer Science and Economics

Expected May 2028

Ann Arbor, MI

- **GPA:** 3.66 / 4.0
- **Coursework:** Data Structures & Algorithms, Intro to Statistics and Data Analysis, Microeconomics, Macroeconomics, Discrete Mathematics, Calculus III, Physics (Mechanics)

## EXPERIENCE

### Michigan Data Consulting (MDC)

Jan 2026 – May 2026

Data Engineer — Michigan Campaign Finance Network

Ann Arbor, MI

- Replaced manual Michigan campaign-finance research — portal searches capped by the Bureau of Elections, irregular Excel exports, hand normalization at ~2 hours per committee — with a Requests + Pandas ETL that ingests filings directly, eliminating ~800 hours of manual pulls across 400 tracked PACs.
- Delivered a production Flask REST API on AWS EC2 as the sole engineer on a 5-month MCFN contract, wiring ingested data and PAC rankings into the nonprofit's public-facing research workflow.

### Vylet | *Python, LangGraph, Redis, Celery, Docker*

May 2026 – Present

Founder — Automated lead-sourcing platform for PE/search-fund; live product generating \$1,500 MRR across three paying clients

[vyletdata.com](https://vyletdata.com)

- Launched Vylet, automating a ~30-minute manual process per business into a Dockerized LangGraph pipeline generating 30 scored leads in 30 minutes — a 30x speedup — with Redis/Celery workers on a recurring cycle.
- Diagnosed a name-collision defect in ownership-verification logic that was incorrectly rejecting valid acquisition targets sharing a name with an unrelated business elsewhere; the fix lifted the pipeline's lead-qualification rate from 79% to 89% with zero change to sourcing volume.

## PROJECTS

### Granular Synthesizer Plugin | *C++, JUCE*

[github.com/Verdent06/granular-synth](https://github.com/Verdent06/granular-synth)

Real-time audio DSP plugin built from scratch in C++/JUCE — sine oscillator through full granular engine with lock-free threading and release-ready VST3/AU binaries

- Enforced the audio thread's zero-allocation constraint by pre-allocating a `MemoryPool<Grain, 64>` slab per voice at plugin startup — a fixed free-list that acquires and releases grain slots without touching the heap — so `processBlock()` never calls `new` or `delete` after `prepareToPlay()` completes.
- Wired UI-to-audio delivery through a hand-rolled SPSC FIFO (64-slot fixed array, atomic head/tail with acquire/release ordering) so slider changes reach the audio thread without a mutex — and routed WAV handoff via atomic `shared_ptr` swap so the UI thread never blocks `processBlock()`.
- Configured CMake to compile VST3 and AU bundles from a single JUCE codebase — targeting a macOS universal binary (arm64 + x86\_64) for native Apple Silicon and Intel — and audited every `processBlock()` path against a real-time safety checklist confirming zero heap allocations and zero lock acquisitions before release builds.

### SignalWeaver | *Python, PostgreSQL, pgvector, Docker, GitHub Actions*

Multi-signal financial research platform combining fundamentals, macro indicators, and news sentiment (research assistant; not investment advice)

- Implemented semantic search over stored financial news with `pgvector` cosine similarity (768-d MPNet embeddings), hitting 49ms p50 / 99ms p99 over 90 queries returning top-5 or top-10 matches.
- Built a React/TypeScript dashboard for composite scores and fundamental/sentiment/macro breakdowns; used it to analyze 90 distinct tickers with scores persisted to Postgres for history views.
- Containerized the stack with Docker Compose (API + Postgres/pgvector + nginx frontend) and a GitHub Actions CI pipeline (frontend build, pytest, API image build on main).

## TECHNICAL SKILLS

Languages ..... Python, C++, SQL

Frameworks ..... Flask, JUCE

Databases ..... PostgreSQL, Redis, pgvector